DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

Taller:

Guía 1

PlantUML, CU, Analisis, CRUD Estudiante

Grupo

Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 28/04/2026

ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA **TALLER 2** **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

1. Objetivo

Modelar el diagrama de casos de uso de análisis del CRUD de Estudiante en PlantUML, partiendo de los requisitos funcionales de nivel 0 y aplicando las relaciones <<*include*>> justificadas, con el fin de establecer la trazabilidad entre requisitos, actores, casos de uso y criterios de aceptación, como base para el posterior análisis estructural y dinámico.

2. Desarrollo

2.1. Requisitos funcionales

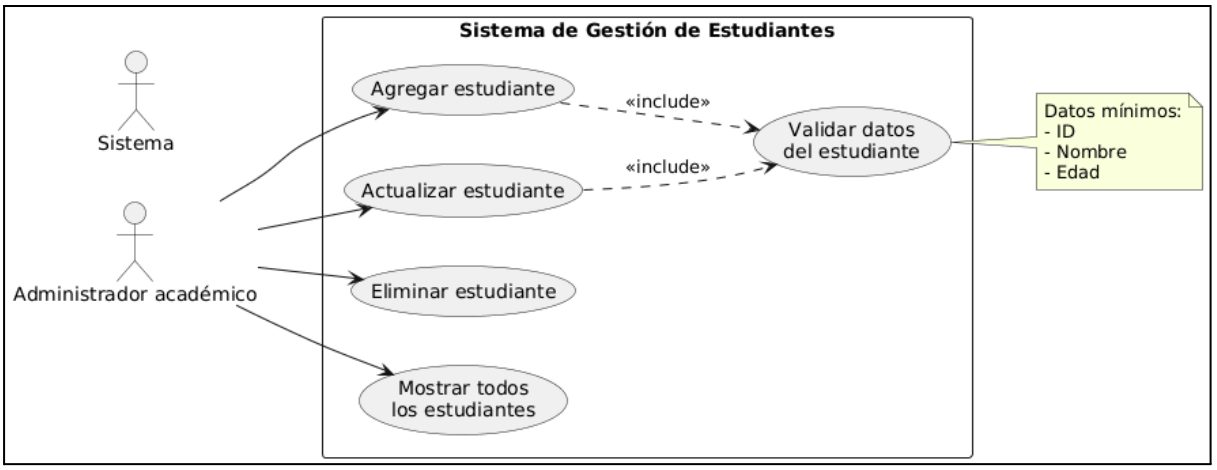
Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir agregar un nuevo estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente usando ID, Nombre y Edad.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante seleccionado cuando corresponda según la regla académica definida.	Administrador académico	Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados en una tabla con las columnas ID, Nombre y Edad.	Administrador académico	Mostrar todo
RF-05	El sistema debe validar los datos obligatorios antes de guardar o actualizar cambios.	Sistema	Validar datos del estudiante

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.2. Diagrama de casos de uso

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema de Gestión de Estudiantes" {
  usecase "Agregar estudiante" as UC1
  usecase "Actualizar estudiante" as UC2
  usecase "Eliminar estudiante" as UC3
  usecase "Mostrar todos\nlos estudiantes" as UC4
  usecase "Validar datos\ndel estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

note right of UC5
Datos mínimos:
- ID
- Nombre
- Edad
end note
@enduml
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick



Especificación breve:

El diagrama muestra dos actores: el Administrador académico, que puede agregar, actualizar, eliminar y mostrar estudiantes; y el Sistema, encargado de validar los datos. Al agregar y actualizar siempre se pasa por una validación obligatoria con <<include>> para verificar que el ID, nombre y edad sean válidos, mientras que eliminar y mostrar todos no necesitan esta validación.

2.3. Puente

```
@startuml
 left to right direction

 actor "Administrador académico" as Admin
 boundary "Pantalla CRUD Estudiantes" as UI
 control "Control Estudiante" as Ctrl
 entity "Estudiante" as Est

 Admin --> UI : selecciona Agregar/Actualizar/Eliminar/Mostrar Todo
 UI --> Ctrl : envía ID, nombre y edad
 Ctrl --> Est : gestiona ID, nombre, edad
 Ctrl --> UI : muestra confirmación o listado
 @enduml
```

Especificación breve:

El diagrama puente conecta el diagrama de casos de uso con el análisis, mostrando cómo el administrador académico interactúa con la pantalla CRUD para ejecutar las operaciones de agregar, actualizar, eliminar o mostrar estudiantes, la pantalla envía los datos al Control Estudiante quien los gestiona con la entidad Estudiante y retorna una confirmación o listado, sirviendo así como base para construir el diagrama de clases de análisis.

2.4. Explicación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Agregar estudiante	Administrador académico	El estudiante no debe existir previamente con el mismo ID.	Ingresar ID, Nombre y Edad -> validar datos -> guardar registro.	Estudiante agregado correctamente.
Actualizar estudiante	Administrador académico	El estudiante debe existir.	Ingresar o seleccionar ID -> modificar Nombre y/o Edad -> validar datos -> guardar cambios.	Datos del estudiante actualizados.
Eliminar estudiante	Administrador académico	El estudiante debe existir y cumplir regla para eliminación.	Seleccionar o ingresar ID -> confirmar eliminación -> eliminar registro.	Estudiante eliminado o marcado como eliminado.
Mostrar todo	Administrador académico	Debe existir al menos un registro o una tabla vacía disponible.	Seleccionar Mostrar Todo -> mostrar registros -> presentar tabla.	Listado visible con ID, Nombre y Edad.

ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA **TALLER 2** **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.5. Matriz de trazabilidad

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único,Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

3. Conclusión técnica

- La implementación de esta guía nos permitió ver claramente cómo el diagrama de casos de uso se relaciona directamente con el diagrama de clases, ya que cada caso de uso cómo agregar, actualizar, eliminar y mostrar se convirtió en métodos concretos dentro de las clases FormularioCrudEstudiante, ControlEstudiante, Estudiante y RepositorioEstudiante.
- Del diagrama de clases surgieron los diagramas de secuencia, donde cada mensaje enviado entre objetos debía corresponder exactamente a un método ya definido en las clases, lo que nos obligó a ser precisos desde el inicio con los nombres y tipos de datos para no generar inconsistencias entre modelos.
- En conclusión, los tres modelos forman una cadena donde el diagrama de casos de uso alimenta al diagrama de clases y este alimenta a los diagramas de secuencia, comprendiendo que modificar uno implica revisar los demás para mantener la trazabilidad del sistema.

4. Recomendaciones

Para futuros talleres de modelado con PlantUML se recomienda compilar el código en paralelo con la escritura para detectar errores de sintaxis en los estereotipos y flechas antes de generar las capturas finales.

Se sugiere revisar con más detalle los criterios de aceptación de cada requisito funcional antes de dibujar el diagrama, pues esto ayuda a decidir si una relación <<include>> es realmente necesaria o no.

También es útil mantener una lista de verificación (actor, límite del sistema, nombres de casos de uso como verbos) impresa al lado mientras se construye el diagrama, para no omitir elementos que la rúbrica evalúa.

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

Taller:

Guía 2

Casos de uso, clases y secuencia CRUD

Grupo

Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 28/04/2026

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

1. Objetivo

Construir los diagramas de clases de análisis y de secuencia para el CRUD de Estudiante en PlantUML, partiendo del diagrama de casos de uso previamente modelado, con el fin de mantener la trazabilidad entre los modelos funcional, estructural y dinámico, y verificar que la colaboración entre objetos (frontera, control, entidad y repositorio) responda a los requisitos funcionales definidos.

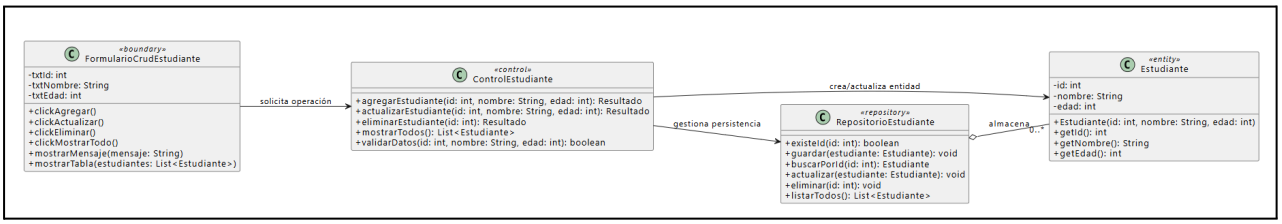
2. Desarrollo

2.1. Diagrama de clases de análisis

```
@startuml
 skinparam classAttributeIconSize 0
 skinparam shadowing false
 left to right direction

 class "FormularioCrudEstudiante" <<boundary>> {
   - txtId: int
   - txtNombre: String
   - txtEdad: int
   + clickAgregar()
   + clickActualizar()
   + clickEliminar()
   + clickMostrarTodo()
   + mostrarMensaje(mensaje: String)
   + mostrarTabla(estudiantes: List<Estudiante>)
 }

 class "ControlEstudiante" <<control>> {
   + agregarEstudiante(id: int, nombre: String, edad: int): Resultado
   + actualizarEstudiante(id: int, nombre: String, edad: int): Resultado
   + eliminarEstudiante(id: int): Resultado
   + mostrarTodos(): List<Estudiante>
   + validarDatos(id: int, nombre: String, edad: int): boolean
 }

 class "Estudiante" <<entity>> {
   - id: int
   - nombre: String
   - edad: int
   + Estudiante(id: int, nombre: String, edad: int)
   + getId(): int
   + getNombre(): String
   + getEdad(): int
 }
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
class "RepositorioEstudiante" <<repository>> {
  + existeId(id: int): boolean
  + guardar(estudiante: Estudiante): void
  + buscarPorId(id: int): Estudiante
  + actualizar(estudiante: Estudiante): void
  + eliminar(id: int): void
  + listarTodos(): List<Estudiante>
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml
```



Especificación breve:

- El diagrama organiza el CRUD de Estudiante en cuatro estereotipos de análisis.
- FormularioCrudEstudiante (<<boundary>>) representa la interfaz visual, con campos txtId (int), txtNombre (String), txtEdad (int) y métodos para cada botón (agregar, actualizar, eliminar, mostrar todo), más mostrar mensaje y mostrar tabla.
- ControlEstudiante (<<control>>) coordina la lógica de los casos de uso e incluye métodos para agregar, actualizar, eliminar, listar todos y validar datos.
- Estudiante (<<entity>>) modela la información del dominio con atributos id, nombre, edad, su constructor y los getters correspondientes.
- RepositorioEstudiante (<<repository>>) gestiona la persistencia, ofreciendo operaciones para verificar existencia, guardar, buscar por ID, actualizar, eliminar y listar todos los estudiantes.
- Las relaciones muestran que la frontera solicita operaciones al control, el control crea o actualiza la entidad y delega la persistencia al repositorio, y el repositorio almacena una colección de cero o más estudiantes.

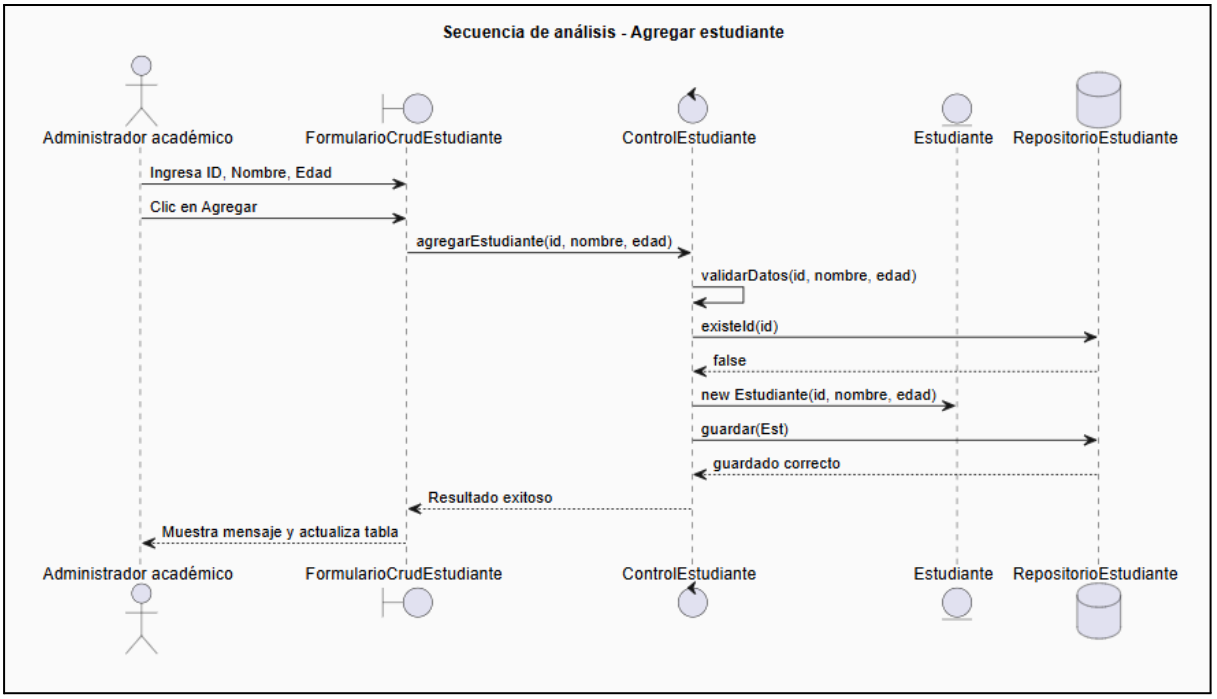
2.2. Diagramas de secuencia

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.2.1. Secuencia: Agregar estudiante

```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick



Especificación breve:

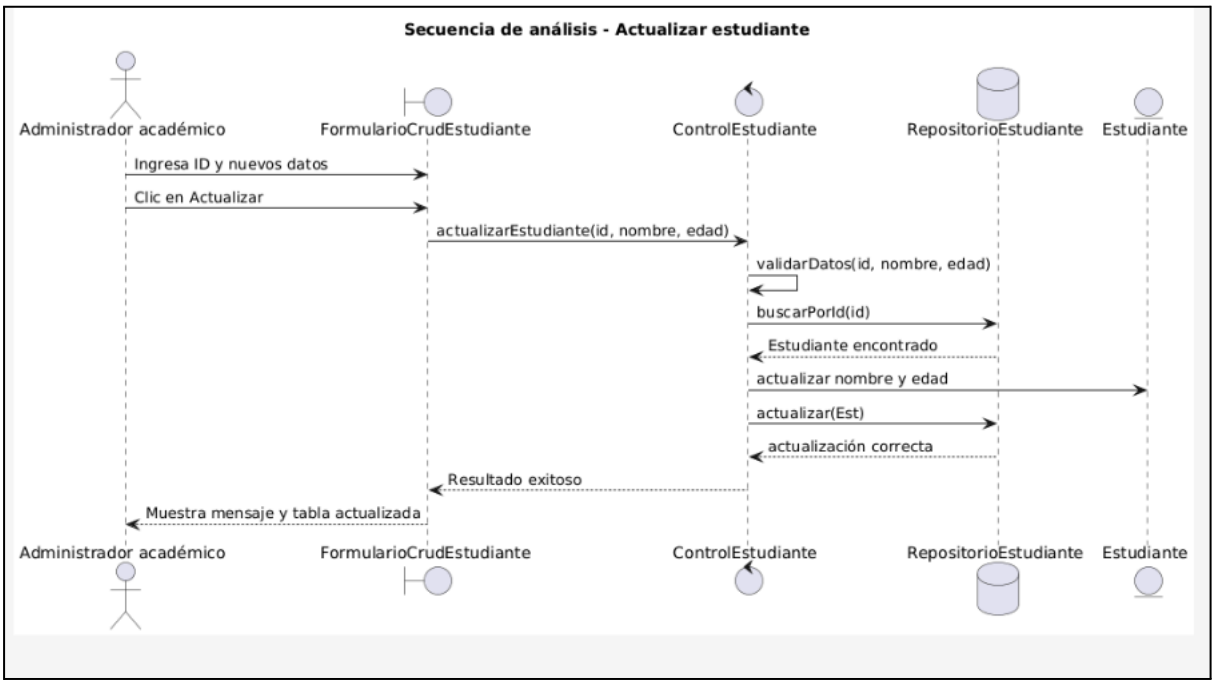
- El Administrador académico ingresa el ID, el Nombre y la Edad en el formulario y hace clic en "Agregar".
- El controlador (ControlEstudiante) recibe la solicitud, valida los datos ingresados y consulta al repositorio si ese ID ya existe.
- Al confirmar que el ID no existe (retorna *false*), el controlador crea un nuevo objeto "Estudiante" con los datos proporcionados.
- Finalmente, se envía la orden de guardar al repositorio, y tras la confirmación, la interfaz muestra un mensaje de éxito y actualiza la tabla visible para el usuario.

2.2.2. Secuencia: Actualizar estudiante

```
@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```



Especificación breve:

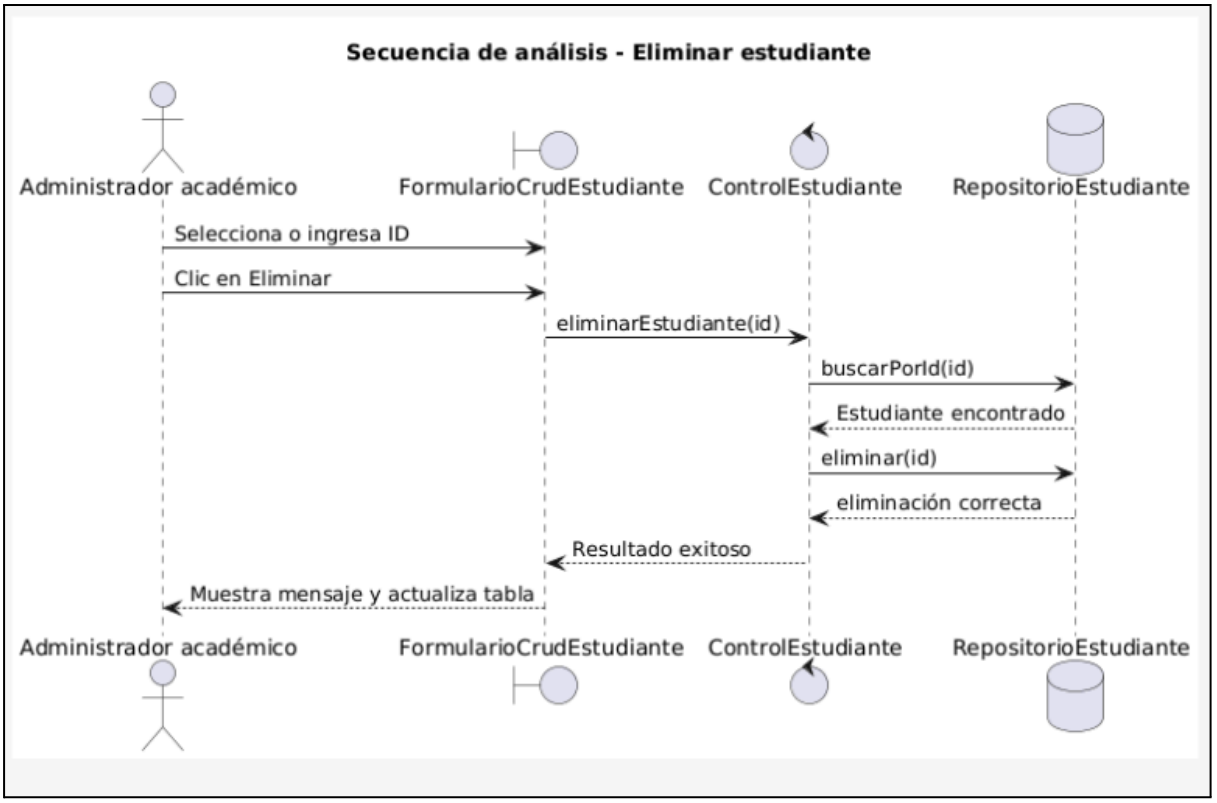
- El Administrador ingresa un ID existente junto con los nuevos datos y hace clic en "Actualizar".

- El controlador valida la información y procede a buscar al estudiante específico en el repositorio utilizando su ID.
- Una vez que el repositorio confirma que el estudiante fue encontrado, el controlador actualiza el nombre y la edad directamente en la entidad.
- Se envía la instrucción al repositorio para guardar esta actualización; al finalizar, la interfaz refleja los cambios en la tabla y muestra un mensaje de éxito.

2.2.3. Secuencia: Eliminar estudiante

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick



Especificación breve:

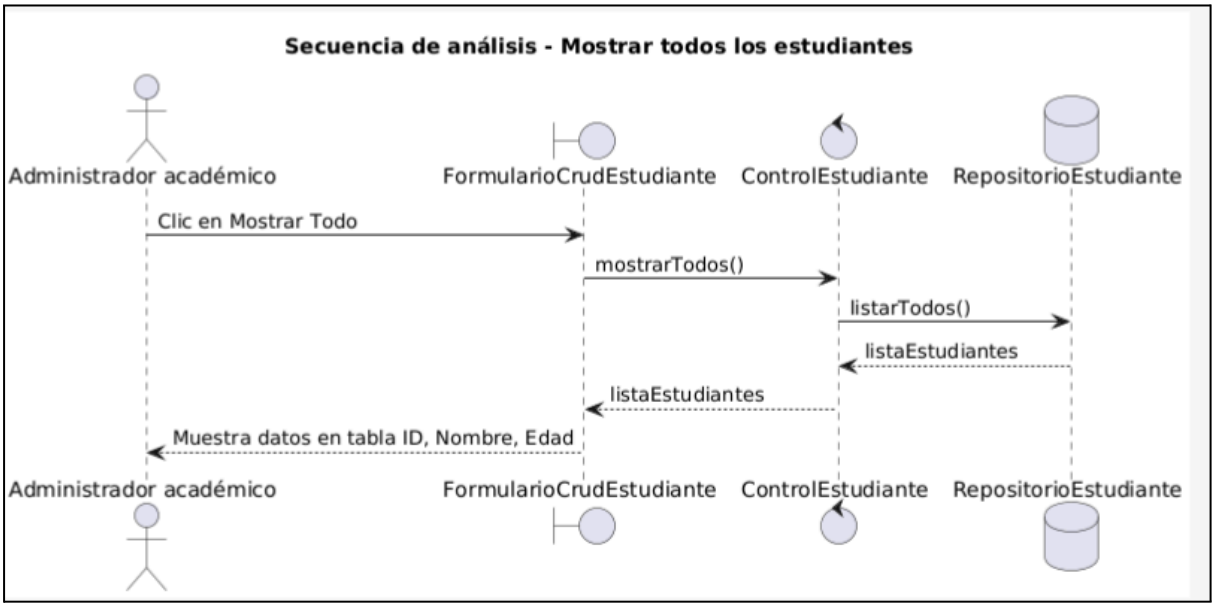
- El Administrador académico selecciona o escribe el ID del estudiante que desea borrar y hace clic en "Eliminar".
- El controlador solicita al repositorio buscar ese ID para asegurarse de que el registro existe.
- Cuando el repositorio confirma que el estudiante fue encontrado, el controlador ejecuta la orden de eliminación.
- El repositorio confirma que la eliminación fue correcta, y la interfaz notifica al administrador actualizando la tabla de datos.

2.2.4. Secuencia: Mostrar todos los estudiantes

```
@startuml
 title Secuencia de análisis - Mostrar todos los estudiantes
 actor "Administrador académico" as Admin
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```



Especificación breve:

- El Administrador académico simplemente hace clic en el botón "Mostrar Todo" dentro de la interfaz.
- El formulario solicita al controlador que muestre a todos los usuarios, y este a su vez le pide al repositorio que liste todos los registros.
- El repositorio devuelve una lista completa de estudiantes al controlador.
- Esta lista viaja de regreso a la interfaz, la cual se encarga de mostrar todos los datos estructurados en las columnas de ID, Nombre y Edad para el administrador.

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.3. Trazabilidad

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único,Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

TALLER 2 UNIDAD 1	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

3. Conclusión técnica

La guía permitió conectar el diagrama de casos de uso con el diagrama de clases de análisis y con los diagramas de secuencia. Cada caso de uso (Agregar, Actualizar, Eliminar, Mostrar todo) se tradujo en responsabilidades asignadas a las clases FormularioCrudEstudiante (frontera), ControlEstudiante (control), Estudiante (entidad) y RepositorioEstudiante (repositorio). A su vez, cada operación CRUD se modeló en un diagrama de secuencia verificando que todos los mensajes correspondieran a métodos existentes en las clases. Esta trazabilidad asegura que los diagramas no son piezas aisladas, sino un modelo coherente que puede servir como base para implementar un programa funcional CRUD de estudiantes en cualquier IDE, cumpliendo con lo solicitado por la ingeniera.

4. Recomendaciones

Para próximos talleres se recomienda unificar los tipos de datos de los atributos desde el inicio, en especial el identificador, decidiendo si será entero o cadena, y mantener esa decisión en todas las clases (entidad, repositorio, control) y en los parámetros de los mensajes de secuencia, evitando mezclas como id: int en el atributo y getId(): String en el getter.

Se sugiere compilar el diagrama de clases de análisis antes de elaborar los diagramas de secuencia, revisando que los nombres de los métodos y los tipos de parámetros estén correctamente escritos, porque cualquier error en la clase se arrastra a las secuencias y genera inconsistencias difíciles de rastrear.

También es recomendable documentar en una tabla aparte la correspondencia entre cada mensaje de secuencia y el método de la clase destino, como una lista de verificación adicional, para garantizar la trazabilidad sin necesidad de rehacer diagramas al final.